

AI ASSISTED CODING

ASSIGNMENT-7.5

NAME: Varshitha. G

HTNO: 2303A51103

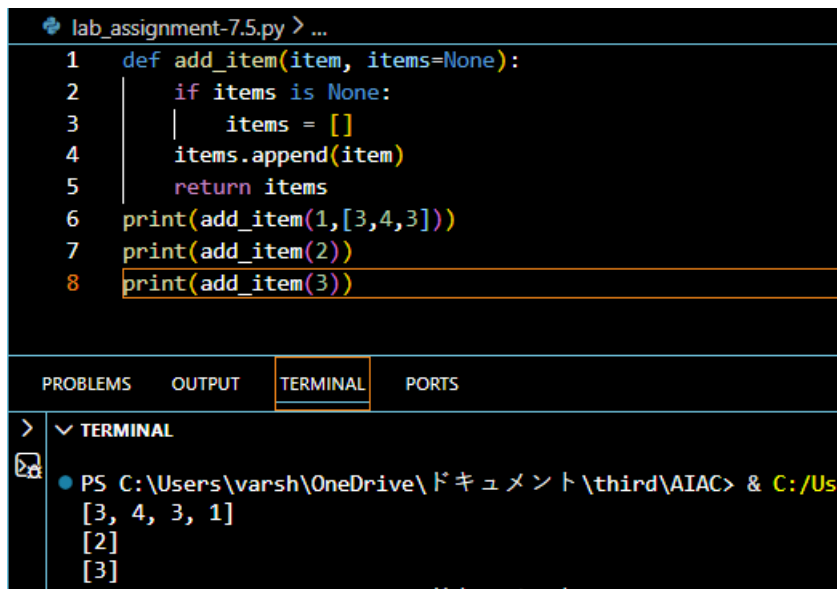
Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):  
    items.append(item)  
    return items  
  
print(add_item(1))  
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.



```
lab_assignment-7.5.py > ...  
1 def add_item(item, items=None):  
2     if items is None:  
3         items = []  
4         items.append(item)  
5     return items  
6 print(add_item(1,[3,4,3]))  
7 print(add_item(2))  
8 print(add_item(3))  
  
PROBLEMS OUTPUT TERMINAL PORTS  
  
> ▼ TERMINAL  
● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Use  
[3, 4, 3, 1]  
[2]  
[3]
```

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

Bug: Floating point precision issue

```
def check_sum():
    return (0.1 + 0.2) == 0.3
print(check_sum())
```

Expected Output: Corrected function

```

10  import math
11
12  def check_sum():
13      return (0.1 + 0.2) == 0.3
14      return math.isclose(0.1 + 0.2, 0.3)
15  print(check_sum())

```

The screenshot shows a code editor with a dark background. The code is as follows:

```

10  import math
11
12  def check_sum():
13      return (0.1 + 0.2) == 0.3
14      return math.isclose(0.1 + 0.2, 0.3)
15  print(check_sum())

```

Below the code editor, there is a terminal window with the following output:

```

>  TERMINAL
True

```

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix. #Bug: No base case

```
def countdown(n):
    print(n)
    return countdown(n-1)
countdown(5)
```

Expected Output : Correct recursion with stopping condition.

```

16  def countdown(n):
17      print(n)
18      if n > 0:
19          return countdown(n-1)
20  countdown(5)

```

The screenshot shows a code editor with a dark background. The code is as follows:

```

16  def countdown(n):
17      print(n)
18      if n > 0:
19          return countdown(n-1)
20  countdown(5)

```

Below the code editor, there is a terminal window with the following output:

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/OneDrive/ドキュメント/third/AIAC/python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/python313/python.exe
5
4
3
2
1
0
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

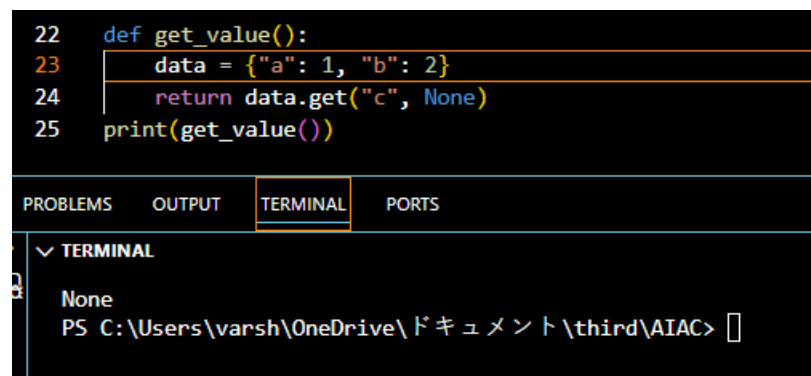
Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():  
    data = {"a": 1, "b": 2}  
    return data["c"]  
print(get_value())
```

Expected Output: Corrected with .get() or error handling.



The screenshot shows a code editor with a dark background. The code is as follows:

```
22 def get_value():  
23     data = {"a": 1, "b": 2}  
24     return data.get("c", None)  
25 print(get_value())
```

Below the code editor, there is a tabbed interface with four tabs: PROBLEMS, OUTPUT, TERMINAL, and PORTS. The TERMINAL tab is selected and highlighted with a yellow border. The terminal output shows the word "None" on the first line, followed by a command prompt "PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> " on the second line.

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():  
    i = 0  
    while i < 5:  
        print(i)
```

Expected Output: Corrected loop increments i.

```
27 def loop_example():
28     i = 0
29     while i < 5:
30         print(i)
31         i += 1 # Increment i
32 loop_example()
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> ✓ **TERMINAL**

4
5 ...

PS C:\Users\varsh\OneDrive\ドキュメント\th
ython313/python.exe c:/Users/varsh/OneDrive

0
1
2
3
4

PS C:\Users\varsh\OneDrive\ドキュメント\th

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

```
3  
4 a, b, _ = (1, 2, 3)
```

PROBLEMS OUTPUT **TERMINAL** PORTS

TERMINAL

ValueError: too many values to unpack (expected 2)

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Lo
ython313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignme

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

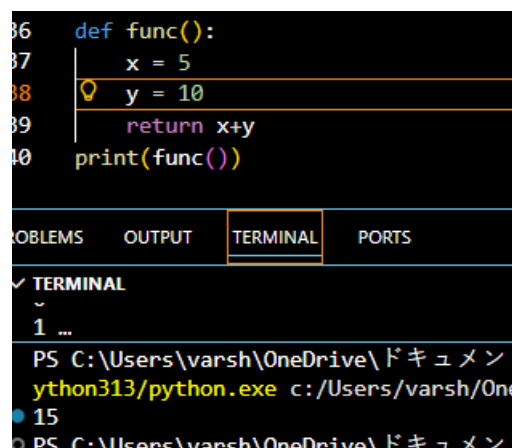
def func():

x = 5

y = 10

return x+y

Expected Output : Consistent indentation applied.



```
36 def func():
37     x = 5
38     y = 10
39     return x+y
40 print(func())
```

PROBLEMS OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

1 ...

PS C:\Users\varsh\OneDrive\ドキュメント\...
python313/python.exe c:/Users/varsh/OneDrive\ドキュメント\...
15

Task 8 (Import Error – Wrong Module Usage)

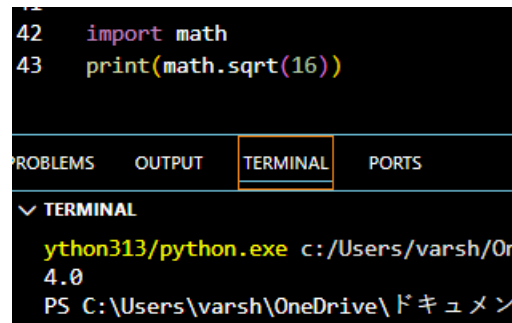
Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
```

```
print(maths.sqrt(16))
```

Expected Output: Corrected to import math



```
42 import math
43 print(math.sqrt(16))
```

PROBLEMS OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

python313/python.exe c:/Users/varsh/OneDrive\ドキュメント\...
4.0

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

```
def total(numbers):
```

```
    for n in numbers:
```

```
        return n
```

```
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns after loop.

```
45 def total(numbers):
46     sum = 0
47     for n in numbers:
48         return n
49     sum += n
50     return sum
51 print(total([1,2,3]))
```

ROBLEMS OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

```
Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC
6
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.

```
52 def calculate_area(length, width):
53     """
54     Calculate the area of a rectangle.
55
56     Args:
57     | length (float): The length of the rectangle.
58     | width (float): The width of the rectangle.
59
60     Returns:
61     | float: The area of the rectangle (length * width).
62
63     Example:
64     | >>> calculate_area(5, 3)
65     | 15
66     """
67     return length * width
68 assert calculate_area(5, 10) == 50
69 assert calculate_area(3, 4) == 12
70 assert calculate_area(7, 7) == 49
71 print(calculate_area(5, 10))
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/
python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/
50
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> □
```

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values():
    return 5 + "10"
print(add_values())
```

Requirements:

- Run the code to observe the error.
- AI should explain why `int + str` is invalid.
- Fix the code by type conversion (e.g., `int("10")` or `str(5)`).
- Verify with 3 assert cases.

Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.

```

3 def add_values():
4     # int + str is invalid because Python cannot add different types
5     # You must convert both to the same type
6     return 5 + int("10")
7
8 assert add_values() == 15
9 assert 5 + int("10") == 15
0 assert int("5") + int("10") == 15
1 print(add_values())

```

BLEMS OUTPUT **TERMINAL** PORTS

TERMINAL

```

Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_ass
15
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list. # Bug: Adding string and list

```
def combine():
```

```
    return "Numbers: " + [1, 2, 3]
```

```
    print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```
142
143 # str + list is invalid because Python cannot concatenate a string with a list directly
144 # Strings and lists are different types, and the + operator doesn't know how to combine the
145 # You must convert the list to a string first using str() or join()
146 def combine():
147     # Fix: Convert list to string using str()
148     return "Numbers: " + str([1, 2, 3])
149 print(combine())
150 # Verify with 3 assert cases
151 assert combine() == "Numbers: [1, 2, 3]", "Test 1 failed"
152 assert isinstance(combine(), str), "Test 2 failed"
153 assert "Numbers:" in combine(), "Test 3 failed"
154 print("All assertions passed!")
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```
Numbers: [1, 2, 3]
All assertions passed!
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Bug: Multiplying string by float

```
def repeat_text():
    return "Hello" * 2.5
print(repeat_text())
```

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases.

```

92 def repeat_text():
93     """
94     Repeats the string "Hello" by multiplying it with an integer.
95
96     This function returns the string "Hello" repeated multiple times. The function
97     converts the float 2.5 to an integer (2) before performing the multiplication.
98
99     Note: In Python, string multiplication requires an integer operand. Float
100     multiplication with strings is invalid because the repetition operation needs
101     a whole number of repetitions. Multiplying a string by a float would be
102     ambiguous - you cannot repeat a string a fractional number of times (e.g.,
103     2.5 times). Therefore, the float must be explicitly converted to an integer
104     using int(), which truncates the decimal part (2.5 becomes 2).
105
106     Returns:
107     |     str: The string "Hello" repeated 2 times, resulting in "HelloHello".
108     """
109     return "Hello" * int(2.5)
110
111 assert repeat_text() == "HelloHello"
112 assert "Hello" * int(2.5) == "HelloHello" # int(2.5) = 2, so "Hello" repeats 2 times
113 assert "Hi" * int(3.7) == "HiHiHi"
114 print(repeat_text())

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**

HelloHello

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer. Bug: Adding None and integer

```
def compute():
```

```
    value = None
```

```
    return value + 10
```

```
print(compute())
```

```

116 def compute():
117     value = 0 # Assign a default value
118     return value + 10
119
120 result = compute()
121 print(result)
122
123 # Validate using asserts
124 assert result == 10, "Test 1 failed"
125 assert isinstance(result, int), "Test 2 failed"
126 assert result > 0, "Test 3 failed"
127 print("All assertions passed!")

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/OneDrive/ドキュメント/third/AIAC/python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/python313/python.exe

10

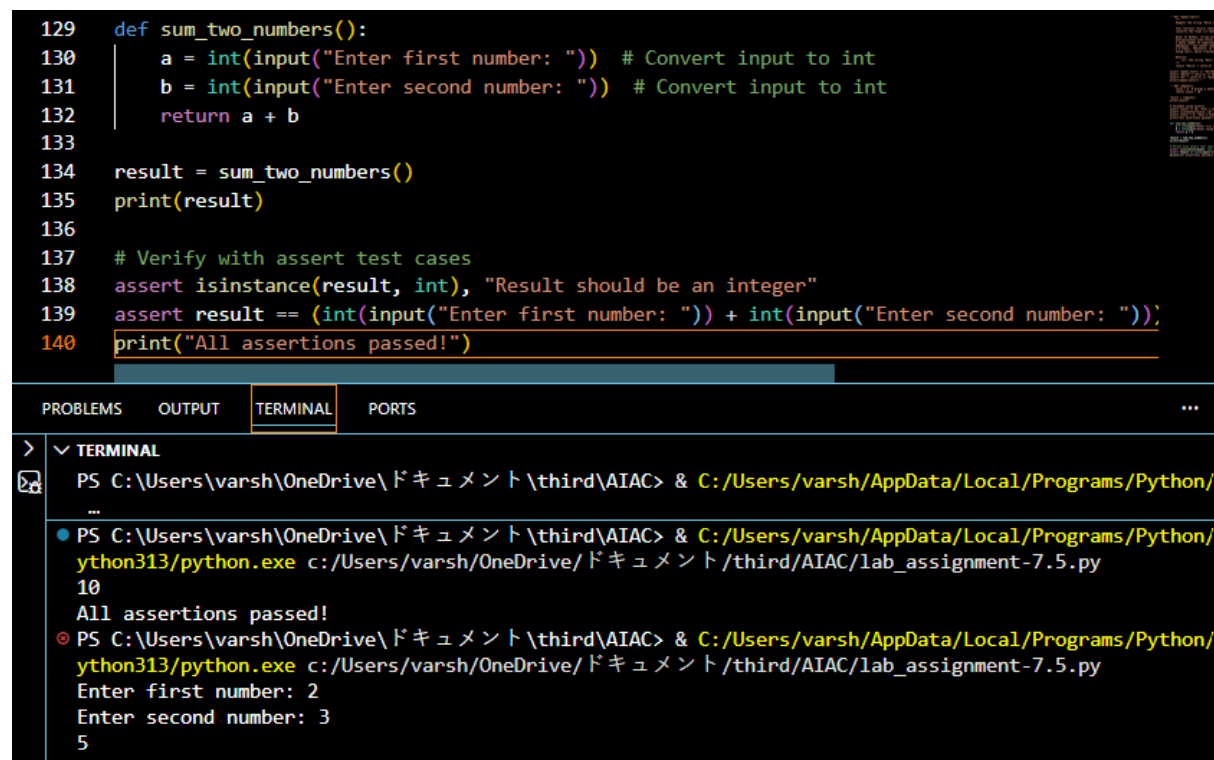
All assertions passed!

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly. Bug: Input remains string

```
def sum_two_numbers():  
    a = input("Enter first number: ")  
    b = input("Enter second number: ")  
    return a + b  
print(sum_two_numbers())
```



The screenshot shows a code editor with a dark background. The code is as follows:

```
129 def sum_two_numbers():  
130     a = int(input("Enter first number: ")) # Convert input to int  
131     b = int(input("Enter second number: ")) # Convert input to int  
132     return a + b  
133  
134 result = sum_two_numbers()  
135 print(result)  
136  
137 # Verify with assert test cases  
138 assert isinstance(result, int), "Result should be an integer"  
139 assert result == (int(input("Enter first number: ")) + int(input("Enter second number: ")))  
140 print("All assertions passed!")
```

Below the code editor is a terminal window with tabs for PROBLEMS, OUTPUT, TERMINAL, and PORTS. The TERMINAL tab is active, showing the following output:

```
> PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-7.5.py  
10  
All assertions passed!  
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-7.5.py  
Enter first number: 2  
Enter second number: 3  
5  
Enter first number: 5
```